

计算机系统结构

第3章 流水线技术

目录

- 3.1 [流水线的基本概念](#)
- 3.2 [流水线的性能指标](#)
- 3.3 [非线性流水线的调度](#)
- 3.4 [流水线的相关与冲突](#)
- 3.5 [流水线的实现](#)

3.4 相关与冲突

3.4 流水线的相关与冲突

3.4.2.2 流水线冲突

流水线冲突(Pipeline Hazard)是指对于具体的流水线来说, 由于相关的存在, 使得指令流中的下一条指令不能在指定的时钟周期执行。

流水线冲突有3种类型:

- ❑ **结构冲突:** 因硬件资源满足不了指令重叠执行的要求而发生的冲突。
- ❑ **数据冲突:** 当指令在流水线中重叠执行时, 因需要用到前面指令的执行结果而发生的冲突。
- ❑ **控制冲突:** 流水线遇到分支指令和其它会改变PC值的指令所引起的冲突。

3.4 流水线的相关与冲突

带来的几个问题：

- 导致错误的执行结果。
- 流水线可能会出现停顿，从而降低流水线的效率和实际的加速比。

基本解决方法：暂停部分指令执行。

当一条指令被暂停时，在该暂停指令之后流出的所有指令都要被暂停，而在该暂停指令之前流出的指令则继续进行（否则就永远无法消除冲突）。

3.4 流水线的相关与冲突

1. 结构冲突

- 在流水线处理机中，为了能够使各种组合的指令都能顺利地重叠执行，需要对功能部件进行流水或重复设置资源。
- 如果某种指令组合因为资源冲突而不能正常执行，则称该处理机有**结构冲突**。
- 常见的导致结构冲突的原因：
 - ❑ 功能部件不是完全流水
 - ❑ 资源份数不够

3.4 流水线的相关与冲突

➤ 结构冲突举例：访存冲突

有些流水线处理机只有一个存储器，将数据和指令放在一起，访存指令会导致访存冲突。

□ 解决办法Ⅰ：插入暂停周期

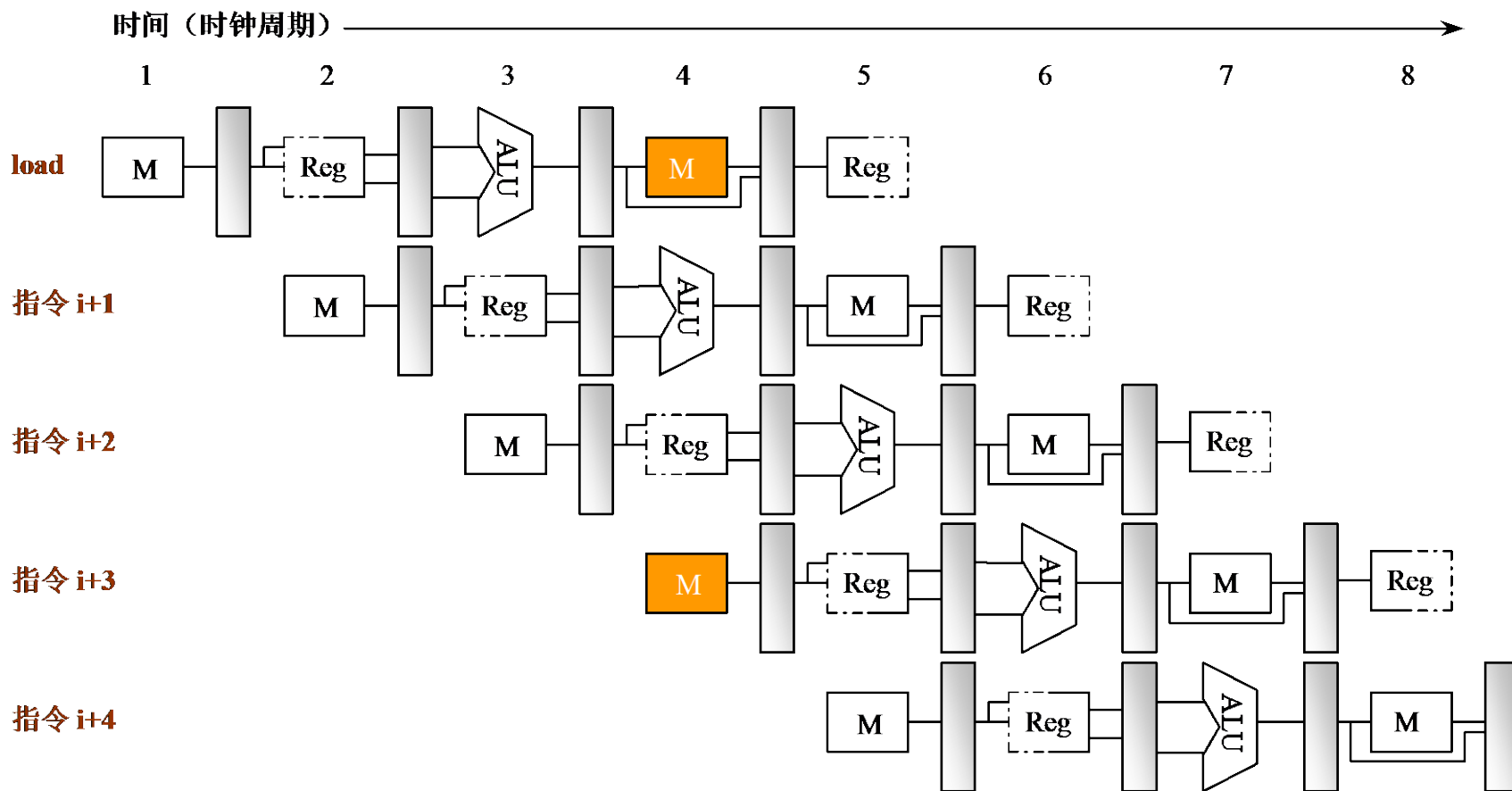
（“流水线气泡”或“气泡”）

引入暂停后的时空图

□ 解决方法Ⅱ：

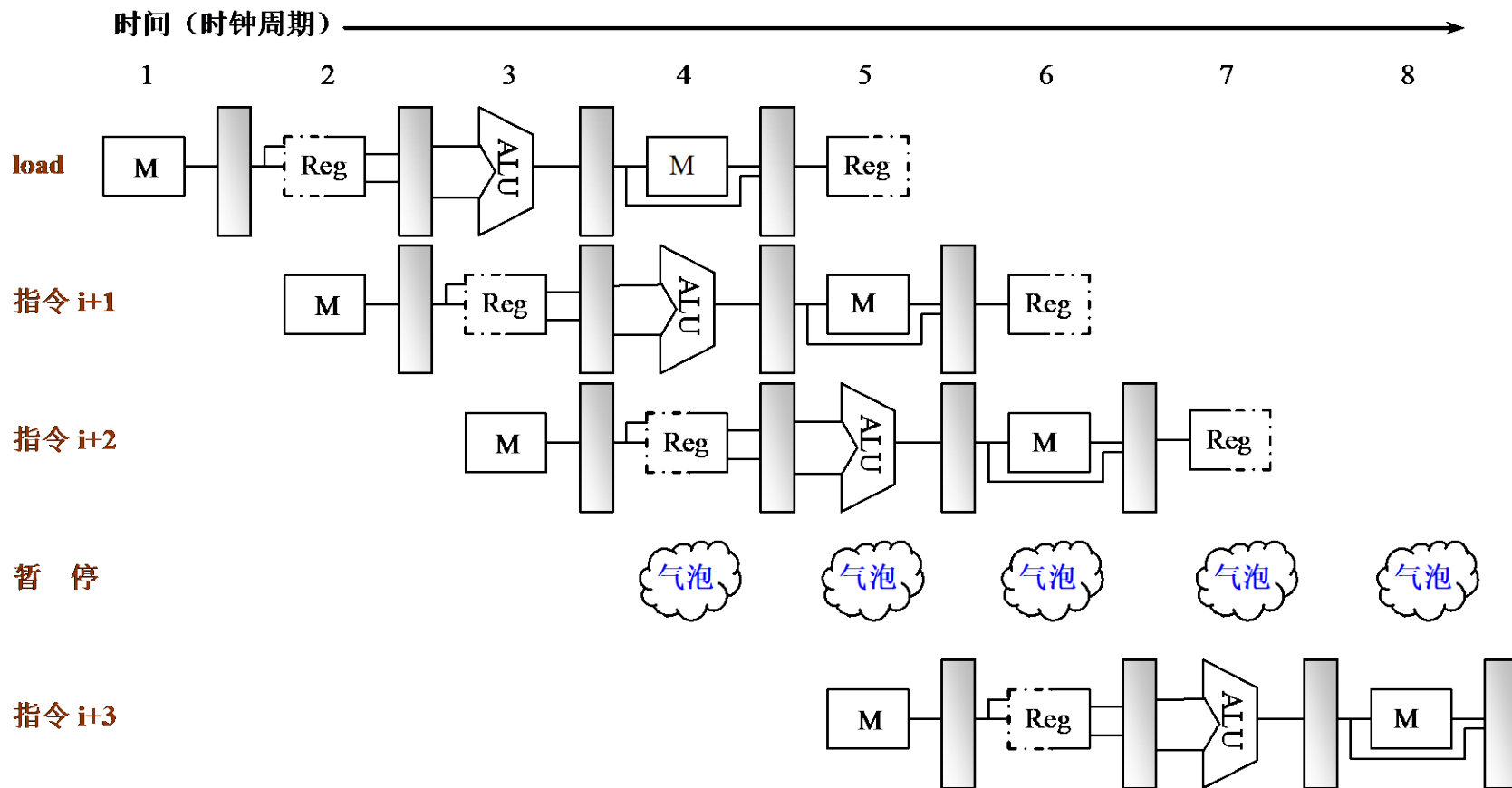
设置相互独立的指令存储器和数据存储器或设置相互独立的指令Cache和数据Cache。

3.4 流水线的相关与冲突



由于访问同一个存储器而引起的结构冲突

3.4 流水线的相关与冲突



为消除结构冲突而插入的流水线气泡

3.4 流水线的相关与冲突

引入暂停后的时空图

指令编号	时钟周期									
	1	2	3	4	5	6	7	8	9	10
指令i	IF	ID	EX	MEM	WB					
指令i+1		IF	ID	EX	MEM	WB				
指令i+2			IF	ID	EX	MEM	WB	WB		
指令i+3				stall	IF	ID	EX	MEM	WB	
指令i+4						IF	ID	EX	MEM	WB
指令i+5							IF	ID	EX	MEM

有时流水线设计者允许结构冲突的存在

主要原因：减少硬件成本

如果把流水线中的所有功能单元完全流水化，或者重复设置足够份数，那么所花费的成本将相当高。

3.4 流水线的相关与冲突

2. 数据冲突

当相关的指令靠得足够近时，它们在流水线中的重叠执行或者重新排序会改变指令读/写操作数的顺序，使之不同于它们串行执行时的顺序，则发生了**数据冲突**。

举例：

DADD R1, R2, R3

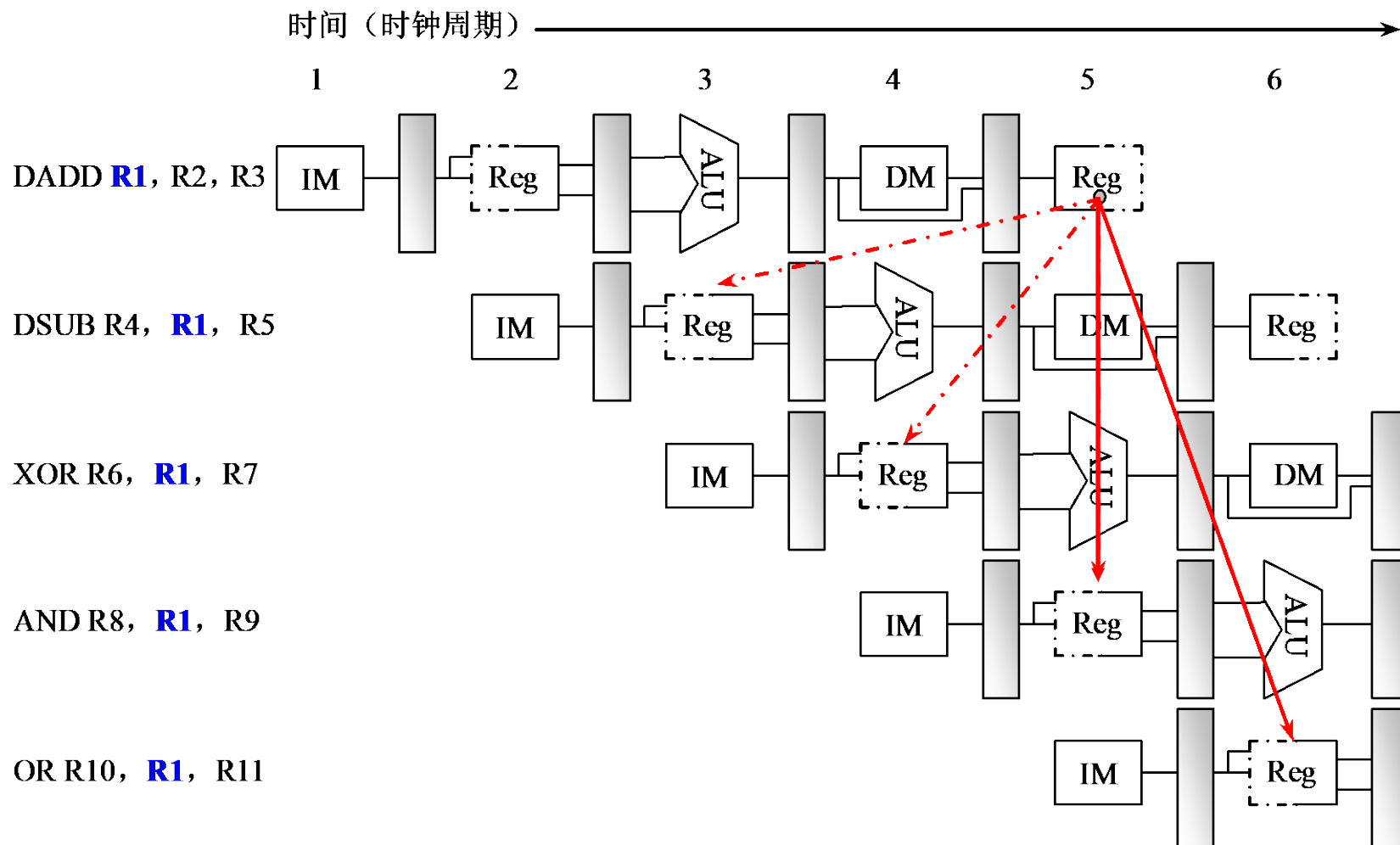
DSUB R4, R1, R5

XOR R6, R1, R7

AND R8, R1, R9

OR R10, R1, R11

3.4 流水线的相关与冲突



流水线的数据冲突举例

3.4 流水线的相关与冲突

- 根据指令读访问和写访问的顺序，可以将数据冲突分为三种类型：（两条指令*i*和*j*，且*i*在*j*之前进入流水线）
 - ❑ 写后读冲突（RAW, Read After Write）

在 *i* 写入之前，*j* 先去读。*j* 读出的内容是错误的。这是最常见的一种数据冲突，它对应于真数据相关。
 - ❑ 写后写冲突（WAW, Write After Write）

在 *i* 写入之前，*j* 先写。最后写入的结果是 *i* 的。这种冲突对应于输出相关。

写后写冲突仅发生在这样的流水线中：

- 流水线中不只一个段可以进行写操作；
- 指令被重新排序了。

前面介绍的5段流水线不会发生写后写冲突。（因只在WB段写寄存器）

3.4 流水线的相关与冲突

□ 读后写冲突（WAR, Write After Read）

在 i 读之前, j 先写。 i 读出的内容是错误的！
由反相关引起。

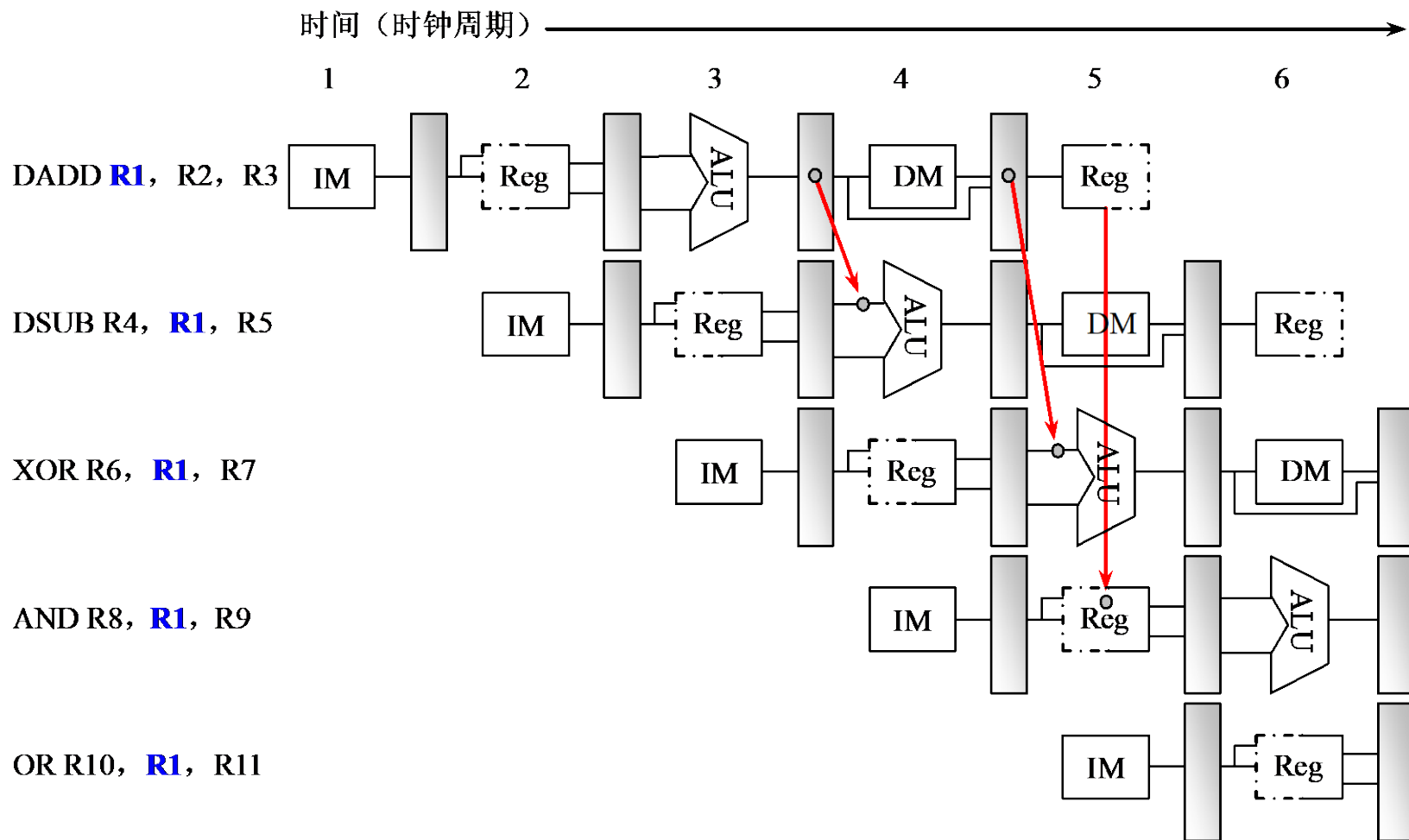
这种冲突仅发生在这样的情况下：

- 有些指令的写结果操作提前了，而且有些指令的读操作滞后了；
- 指令被重新排序了。

3.4 流水线的相关与冲突

- 通过**定向技术（Forwarding）**减少数据冲突引起的停顿（定向技术也称为旁路或短路）
 - ❑ **关键思想：**在计算结果尚未出来之前，后面等待使用该结果的指令并不真正立即需要该计算结果，如果能够将该计算结果从其产生的地方直接送到其它指令需要它的地方，那么就可以避免停顿。
 - ❑ 采用定向技术消除上例中的相关

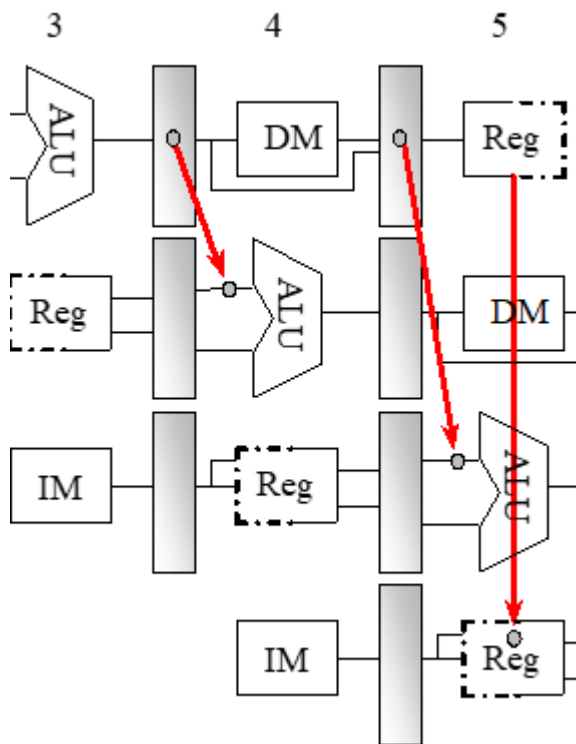
3.4 流水线的相关与冲突



采用定向技术后的流水线数据通路

3.4 流水线的相关与冲突

▣ 定向的实现



- EX段和MEM段之间的流水寄存器中保存的ALU运算结果总是回送到ALU的入口。
- 当定向硬件检测到前一个ALU运算结果写入的寄存器就是当前ALU操作的源寄存器时，那么控制逻辑就选择定向的数据作为ALU的输入，而不采用从通用寄存器组读出的数据。

- ▣ 结果数据不仅可以从某一功能部件的输出定向到其自身的输入，而且还可以定向到其它功能部件的输入。

3.4 流水线的相关与冲突

➤ 需要停顿的数据冲突

- 并不是所有的数据冲突都可以用定向技术来解决。

举例:

LD R1, 0 (R2)

DADD R4, R1, R5

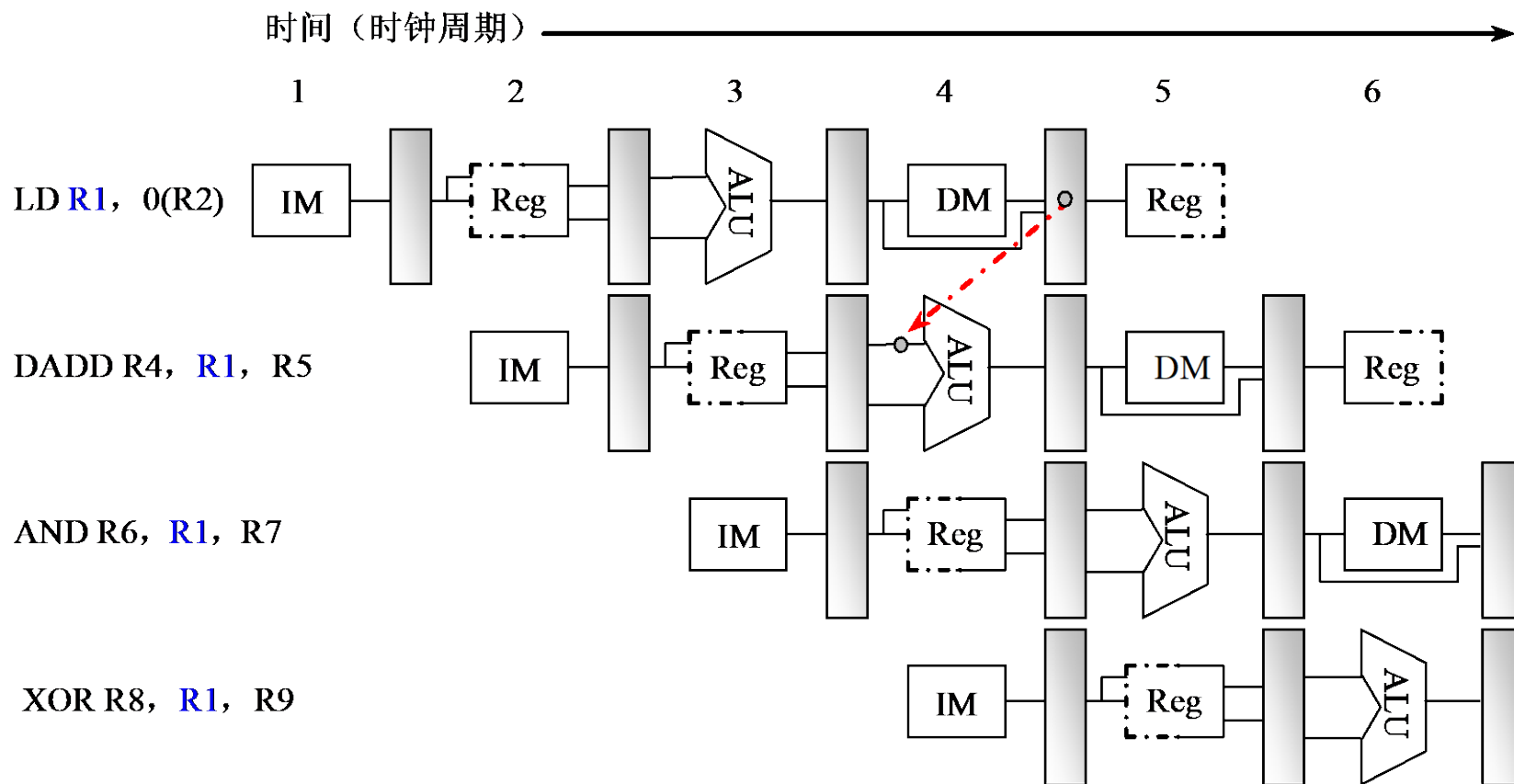
AND R6, R1, R7

XOR R8, R1, R9

- 增加**流水线互锁机制 (Pipeline Interlock)**，插入“暂停”。

作用：检测发现数据冲突，并使流水线停顿，直至冲突消失。

3.4 流水线的相关与冲突



无法将LD指令的结果定向到DADD指令

3.4 流水线的相关与冲突

- 依靠编译器解决数据冲突

让编译器重新组织指令顺序来消除冲突，这种技术称为**指令调度（Instruction Scheduling）**或**流水线调度（Pipeline Scheduling）**。

3.4 流水线的相关与冲突

□ 举例：

请为下列表达式生成没有暂停的指令序列：

$A = B + C$ ；

$D = E - F$ ；

假设载入延迟为1个时钟周期。

调度前的代码		调度后的代码	
停顿	LD Rb, B	LD Rb, B	
	LD Rc, C	LD Rc, C	
	DADD Ra, Rb, Rc	LD Re, E	
	SD Ra, A	DADD Ra, Rb, Rc	
停顿	LD Re, E	LD Rf, F	
	LD Rf, F	SD Ra, A	
	DSUB Rd, Re, Rf	DSUB Rd, Re, Rf	
	SD Rd, D	SD Rd, D	

3.4 流水线的相关与冲突

3. 控制冲突

- 执行分支指令的结果有两种
 - ❑ **分支成功**：PC值改变为分支转移的目标地址。
在条件判定和转移地址计算都完成后，才改变PC值。
 - ❑ **不成功或者失败**：PC的值保持正常递增，
指向顺序的下一条指令。
- 处理分支指令最简单的方法：
“冻结（Freeze）”或者“排空（Flush）”流水线
优点：简单
 - ❑ 前述5段流水线中，改变PC值是在MEM段进行的。
给流水线带来了3个时钟周期的延迟

3.4 流水线的相关与冲突

简单处理分支指令：分支成功的情况

分支指令	IF	ID	EX	MEM	WB					
分支目标指令		IF	stall	stall	IF	ID	EX	MEM	WB	
分支目标指令+1						IF	ID	EX	MEM	WB
分支目标指令+2							IF	ID	EX	MEM
分支目标指令+3								IF	ID	EX

这个5段流水线中，ID段检测到分支指令，暂停其后所有指令的执行，直到MEM段，确定新PC值。

给流水线带来了3个时钟周期的延迟

3.4 流水线的相关与冲突

- 把由分支指令引起的延迟称为分支延迟。
- 分支指令在目标代码中出现的频度

- 每3~4条指令就有一条是分支指令。

假设：分支指令出现的频度是30%

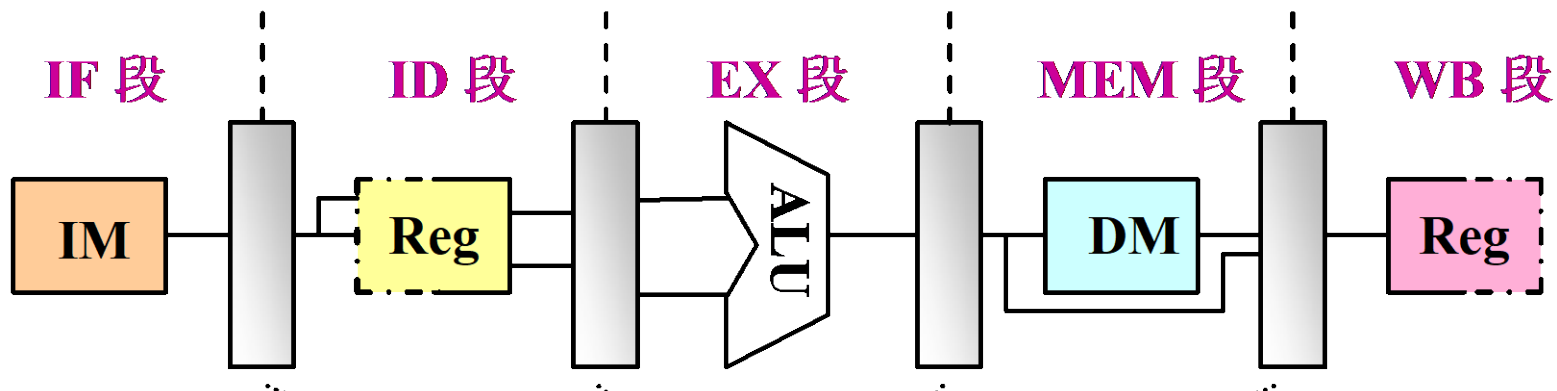
流水线理想 $CPI=1$

那么：流水线的实际

$$CPI = 70\% * 1 + 30\% * (1+3) = 1.9$$

- 可采取两种措施来减少分支延迟。
 - 在流水线中尽早判断出分支转移是否成功；
 - 尽早计算出分支目标地址。

3.4 流水线的相关与冲突



分支指令要做两件事：

判断分支是否成功+分支目标地址

MEM段 =》 ID段

下面的讨论中，我们假设：

这两步工作被提前到ID段完成，即分支指令是在ID段的末尾执行完成，所带来的分支延迟为一个时钟周期。

3.4 流水线的相关与冲突

➤ 3种通过软件（编译器）来减少分支延迟的方法

- (1) 预测分支失败
- (2) 预测分支成功
- (3) 延迟分支

共同点：

- 对分支的处理方法在程序的执行过程中始终是不变的，是静态的。
- 要么总是预测分支成功，要么总是预测分支失败。

3.4 流水线的相关与冲突

➤ 3种通过软件（编译器）来减少分支延迟的方法

（1）预测分支失败

- 允许分支指令后的指令继续在流水线中流动，就好像什么都没发生似的；
- 若确定分支失败，将分支指令看作是一条普通指令，流水线正常流动；
- 若确定分支成功，流水线就把在分支指令之后取出的所有指令转化为空操作，并按分支目地重新取指令执行。

要保证：分支结果出来之前不能改变处理机的状态，以便一旦猜错时，处理机能够回退到原先的状态。

DLX流水线对分支的处理过程

分支指令 i(失败)	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	EX	MEM	WB			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

分支指令 i(成功)	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	idle	idle	idle			
分支目标 j			IF	ID	EX	MEM	WB		
分支目标 j+1				IF	ID	EX	MEM	WB	
分支目标 j+2					IF	ID	EX	MEM	WB

预测分支失败

3.4 流水线的相关与冲突

(2) 预测分支成功

假设分支转移成功，并从分支目标地址处取指令执行。

起作用的前题：先知道分支目标地址，后知道分支是否成功。

前述5段流水线中，由于判断分支是否成功与分支目标地址计算是在同一流水段完成的，所以这种方法没有任何好处。

(3) 延迟分支 (Delayed Branch)

主要思想：

从逻辑上“延长”分支指令的执行时间。把延迟分支看成是由原来的分支指令和若干个延迟槽构成，不管分支是否成功，都要按顺序执行延迟槽中的指令。

具有一个分支延迟槽的流水线的执行过程

分支失败	分支指令 i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	ID	EX	MEM	WB			
	指令 i+2			IF	ID	EX	MEM	WB		
	指令 i+3				IF	ID	EX	MEM	WB	
	指令 i+4					IF	ID	EX	MEM	WB

分支成功	分支指令 i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	ID	EX	MEM	WB			
	分支目标指令 j			IF	ID	EX	MEM	WB		
	分支目标指令 j+1				IF	ID	EX	MEM	WB	
	分支目标指令 j+2					IF	ID	EX	MEM	WB

分支延迟槽中的指令“掩盖”了流水线原来必需插入的暂停周期。

3.4 流水线的相关与冲突

分支延迟指令的调度

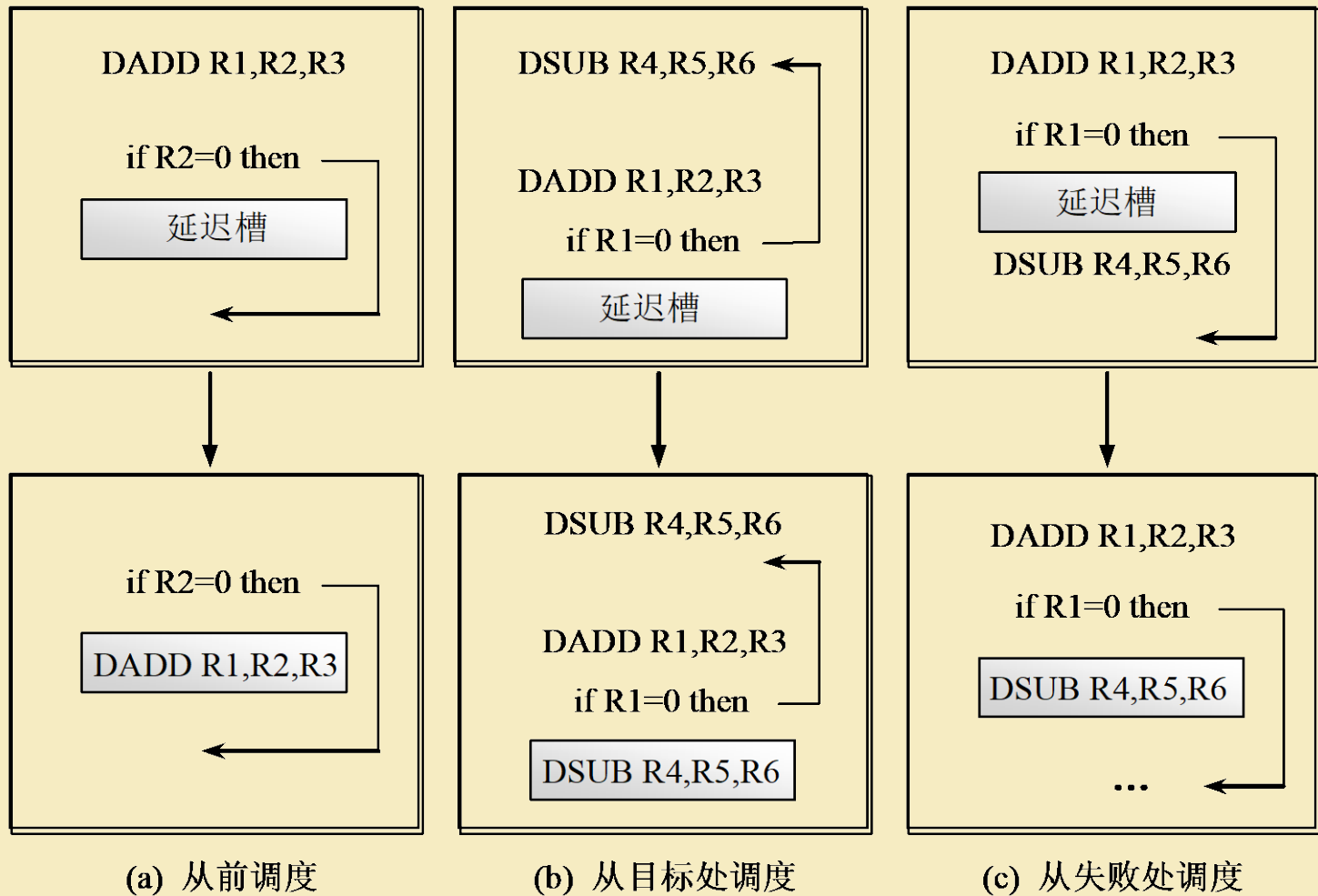
任务：在延迟槽中放入有用的指令

由编译器完成。能否带来好处取决于编译器能否把有用的指令调度到延迟槽中。

三种调度方法：

- 从前调度
- 从目标处调度
- 从失败处调度

调度前和调度后的代码



3.4 流水线的相关与冲突

三种方法的要求及效果

调 度 策 略	对调度的要求	什么情况下起作用？
从 前 调 度	被调度的指令必须与分支无关	任何情况
从目标处调度	必须保证在分支失败时执行被调度的指令不会导致错误。有可能需要复制指令。	分支成功时 (但由于复制指令，有可能会增大程序空间)
从失败处调度	必须保证在分支成功时执行被调度的指令不会导致错误。	分支失败时

3.4 流水线的相关与冲突

分支延迟受到两个方面的限制：

- 可以被放入延迟槽中的指令要满足一定的条件；
- 编译器预测分支转移方向的能力。

进一步改进：分支取消（Canceling or Nullifying）机制

当分支的实际执行方向和事先所预测的一样时，
执行分支延迟槽中的指令，否则就将分支延迟槽中的
指令转化成一个空操作。

3.4 流水线的相关与冲突

预测错误

分支失败	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	idle	idle	idle	idle			
	指令 i+2			IF	ID	EX	MEM	WB		
	指令 i+3				IF	ID	EX	MEM	WB	
	指令 i+4					IF	ID	EX	MEM	WB

预测正确

分支成功	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	ID	EX	MEM	WB			
	分支目标指令j			IF	ID	EX	MEM	WB		
	分支目标指令j+1				IF	ID	EX	MEM	WB	
	分支目标指令j+2					IF	ID	EX	MEM	WB

预测分支成功的情况下，分支取消机制的执行情况

3.5 流水线的实现

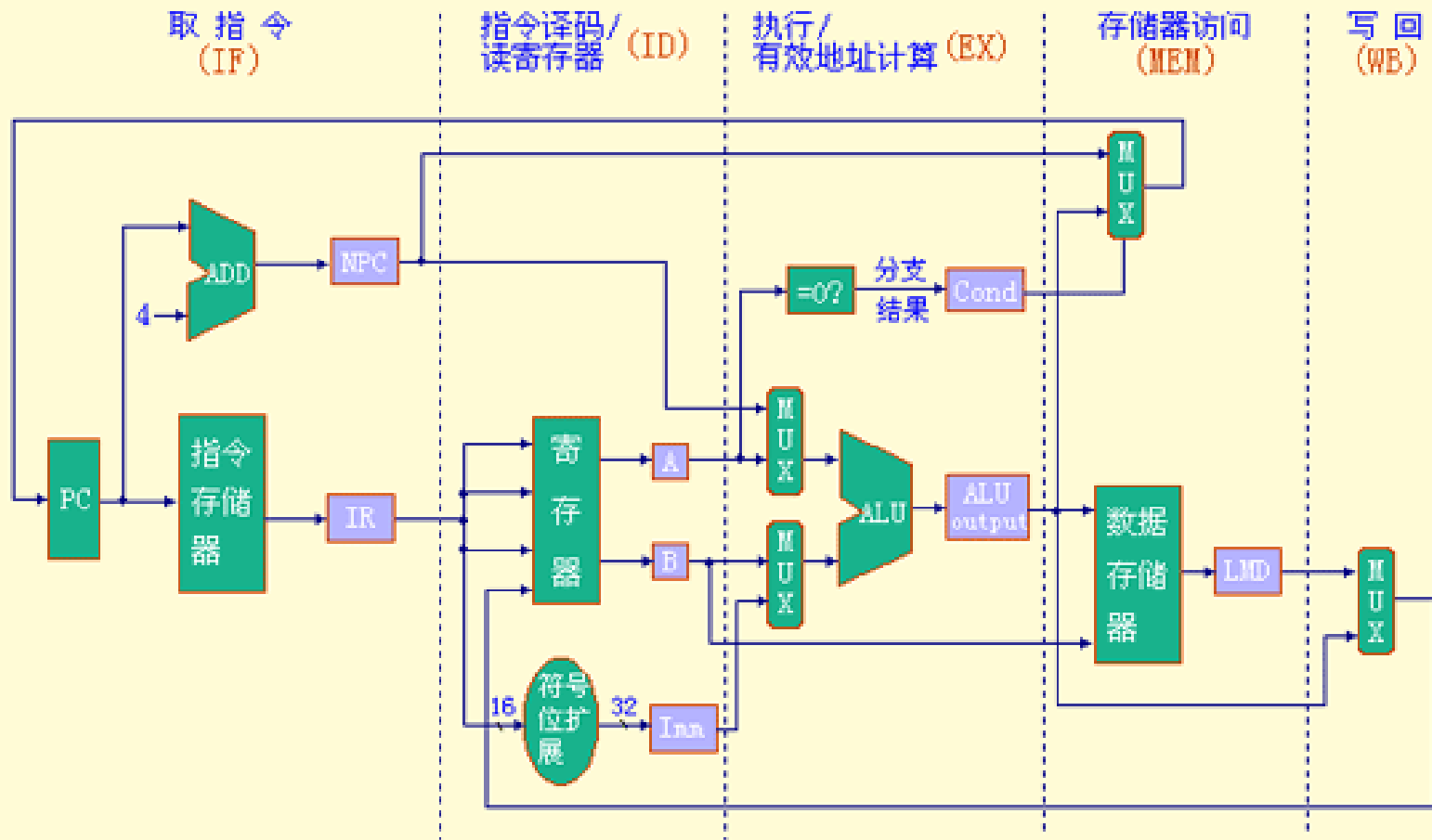
3.5 流水线的实现

3.5.1 MIPS的一种简单实现

1. 实现MIPS指令子集的一种简单数据通路。

- 该数据通路的操作分成5个时钟周期
 - 取指令（IF）
 - 指令译码/读寄存器（ID）
 - 执行/有效地址计算（EX）
 - 存储器访问/分支完成（MEM）
 - 写回（WB）
- 只讨论整数指令的实现
（包括：load和store，等于0转移，整数ALU指令等。）

实现DLX指令的一种简单数据通路



3.5 流水线的实现

- **通用寄存器**：指令之间保存结果。如PC、通用寄存器组、存储器单元等。
- **临时寄存器**：在单条指令的执行过程中保存中间结果。
 - **NPC**：下一条程序计数器，存放下一条指令的地址。
 - **IR**：指令寄存器，存放当前正在处理的指令。
 - **A**：第一操作数寄存器，存放从通用寄存器组读出来的操作数。
 - **B**：第二操作数寄存器，存放从通用寄存器组读出来的另一个操作数。
 - **Imm**：存放符号扩展后的立即数操作数。
 - **Cond**：存放条件判定的结果。为“真”表示分支成功。
 - **ALUo**：存放ALU的运算结果。
 - **LMD**：存放load指令从存储器读出的数据。

3.5 流水线的实现

2. 一条MIPS指令最多需要以下5个时钟周期:

➤ 取指令周期 (IF)

- $IR \leftarrow Mem[PC]$

- $NPC \leftarrow PC + 4$

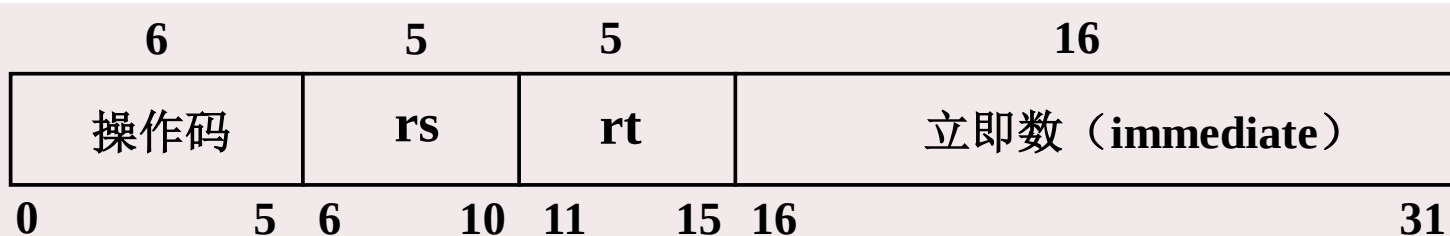
➤ 指令译码/读寄存器周期 (ID)

- $A \leftarrow Regs[rs]$

- $B \leftarrow Regs[rt]$

- $Imm \leftarrow ((IR_{16})^{16} \# IR_{16..31})$ IR的低16位符号扩展

I类指令



指令的译码操作和读寄存器操作是并行进行的。

原因：在MIPS指令格式中，操作码字段以及rs、rt字段都是在固定的位置。这种技术称为**固定字段译码**技术。

3.5 流水线的实现

➤ 执行/有效地址计算周期 (EX)

不同指令所进行的操作不同：

- load指令和store指令

$$ALUo \leftarrow A + Imm$$

- 寄存器—寄存器ALU指令

$$ALUo \leftarrow A \text{ funct } B$$

- 寄存器—立即值ALU指令

$$ALUo \leftarrow A \text{ op } Imm$$

- 分支指令

$$ALUo \leftarrow NPC + (Imm \ll 2) ;$$

$$cond \leftarrow (A == 0)$$

将有效地址计算周期和执行周期合并为一个时钟周期，这是因为MIPS指令集采用load / store结构，没有任何指令需要同时进行数据有效地址的计算、转移目标地址的计算和对数据进行运算。

3.5 流水线的实现

➤ 存储器访问/分支完成周期 (MEM)

- 所有指令都要在该周期对PC进行更新。

除了分支指令，其它指令都是做：PC←NPC

- 在该周期内处理的MIPS指令仅仅有load、store和分支三种指令。

- load指令和store指令

$LMD \leftarrow Mem[ALUo]$

或者 $Mem[ALUo] \leftarrow B$

- 分支指令

if (cond) PC ← ALUo else PC ← NPC

3.5 流水线的实现

➤ 写回周期 (WB)

不同的指令在写回周期完成的工作也不一样。

- 寄存器—寄存器ALU指令

$$\text{Regs}[\text{rd}] \leftarrow \text{ALUo}$$

- 寄存器—立即数ALU指令

$$\text{Regs}[\text{rt}] \leftarrow \text{ALUo}$$

- load指令

$$\text{Regs}[\text{rt}] \leftarrow \text{LMD}$$

3.5 流水线的实现

3.5.2 基本的MIPS流水线

每一个时钟周期完成的工作看作是流水线的一段，每个时钟周期启动一条新的指令。

1. 流水实现的数据通路主要做了以下改动：

➤ 设置了流水寄存器

- 段与段之间设置流水寄存器

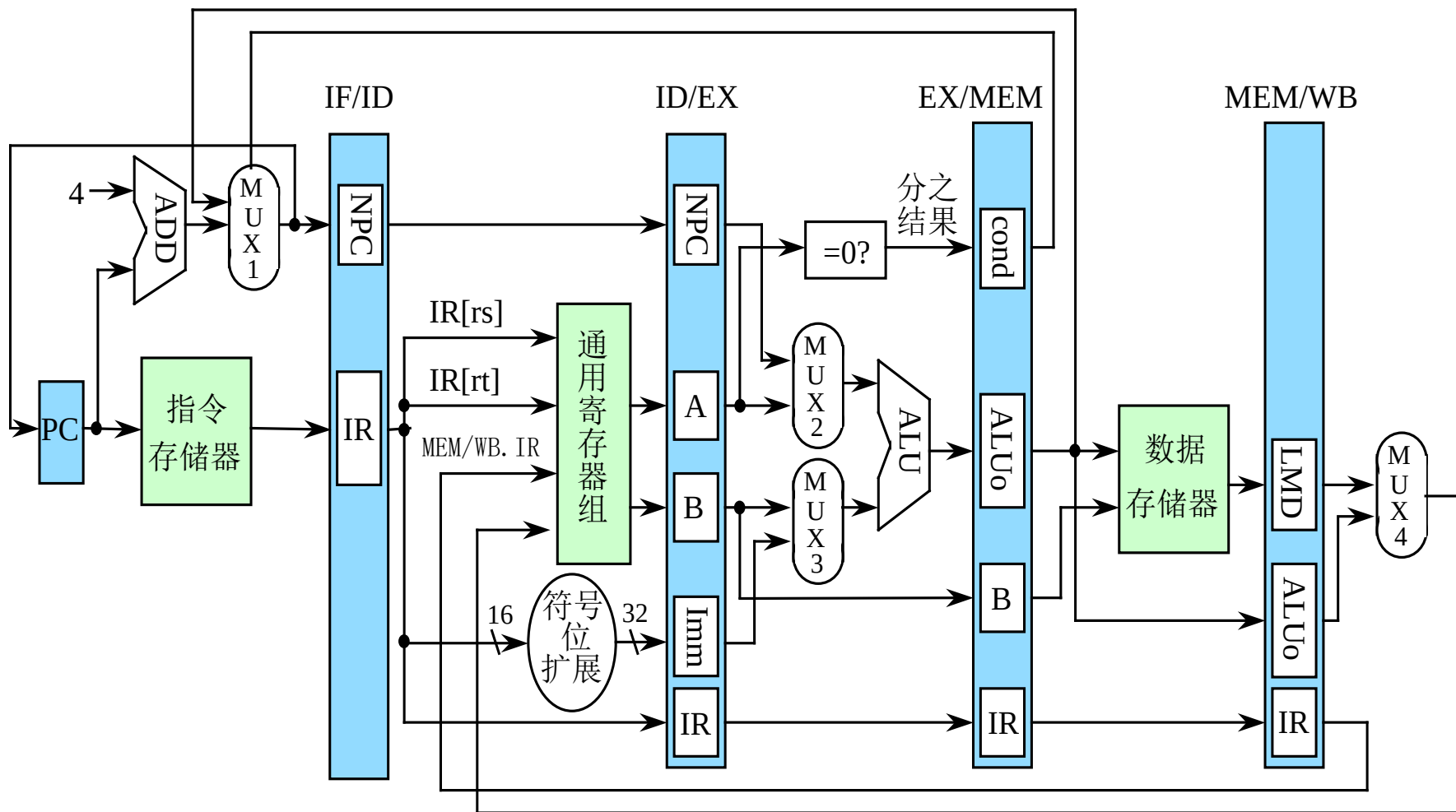
- 流水寄存器的名称

用其相邻的两个段的名称拼合而成。

例如：ID段与EX段之间的流水寄存器用ID/EX表示

- 每个流水寄存器是由若干个寄存器构成的

3.5 流水线的实现



流水实现的数据通路

3.5 流水线的实现

- 寄存器的命名形式为：x.y
- 所包含的字段的命名形式为：x.y[s]

其中：x：流水寄存器名称

y：具体寄存器名称

s：字段名称

例如：

ID/EX. IR：流水寄存器ID/EX中的子寄存器IR

ID/EX. IR[op]：该寄存器的op字段（即操作码字段）

- 流水寄存器的作用
 - 将各段的工作隔开，使得它们不会互相干扰。
 - 保存相应段的处理结果。

3.5 流水线的实现

例如：

EX/MEM. ALUo：保存EX段ALU的运算结果

MEM/WB. LMD：保存MEM段从数据存储器读出的数据

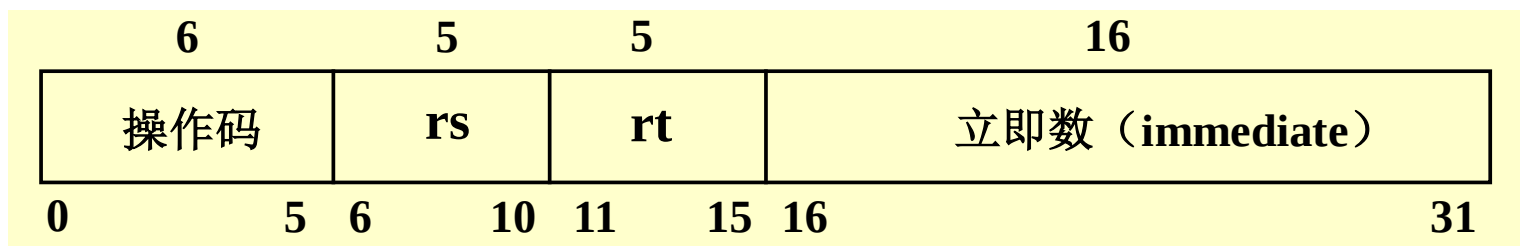
- 向后传递后面将要用到的数据或者控制信息
所有有用的数据和控制信息每个时钟周期
会随着指令在流水线中的流动往后流动一段。

- 增加了向后传递IR和从MEM/WB. IR回送到通用寄存器组的连接。
- 将对PC的修改移到了IF段，以便PC能及时地加4，为取下一条指令做好准备。

3.5 流水线的实现

2. 每一个流水段进行的操作

- $IR[rs] = IR_{6..10}$
- $IR[rt] = IR_{11..15}$
- $IR[rd] = IR_{16..20}$



流水线的每个流水段的操作

流水段	所有指令类型		
IF	$\text{IF/ID. IR} \leftarrow \text{Mem}[\text{PC}]$ $\text{IF/ID. NPC, PC} \leftarrow (\text{if} ((\text{EX/MEM. IR}[\text{op}] == \text{branch}) \& \text{EX/MEM. cond}) \{ \text{EX/MEM. ALUo} \} \text{ else } \{ \text{PC}+4 \}) ;$		
ID	$\text{ID/EX. A} \leftarrow \text{Regs}[\text{IF/ID. IR}[\text{rs}]] ; \text{ID/EX. B} \leftarrow \text{Regs}[\text{IF/ID. IR}[\text{rt}]] ;$ $\text{ID/EX. NPC} \leftarrow \text{IF/ID. NPC} ; \text{ID/EX. IR} \leftarrow \text{IF/ID. IR} ;$ $\text{ID/EX. Imm} \leftarrow (\text{IF/ID. IR}_{16})^{16} \text{##IF/ID. IR}_{16..31} ;$		
EX	ALU 指令	load/store 指令	分支指令
	$\text{EX/MEM. IR} \leftarrow \text{ID/EX. IR} ;$ $\text{EX/MEM. ALUo} \leftarrow$ $\text{ID/EX. A } \textit{funct} \text{ ID/EX. B}$ 或 $\text{EX/MEM. ALUo} \leftarrow$ $\text{ID/EX. A } \textit{op} \text{ ID/EX. Imm} ;$	$\text{EX/MEM. IR} \leftarrow \text{ID/EX. IR} ;$ $\text{EX/MEM. ALUo} \leftarrow$ $\text{ID/EX. A} + \text{ID/EX. Imm} ;$ $\text{EX/MEM. B} \leftarrow \text{ID/EX. B} ;$	$\text{EX/MEM. IR} \leftarrow \text{ID/EX. IR} ;$ $\text{EX/MEM. ALUo} \leftarrow$ $\text{ID/EX. NPC} +$ $\text{ID/EX. Imm} \ll 2 ;$ $\text{EX/MEM. cond} \leftarrow$ $(\text{ID/EX. A} == 0) ;$

流水线的每个流水段的操作

流水段	任何指令类型		
	ALU 指令	load/store 指令	分支指令
MEM	$\text{MEM/WB. IR} \leftarrow \text{EX/MEM. IR};$ $\text{MEM/WB. ALU}_o \leftarrow$ $\text{EX/MEM. ALU}_o;$	$\text{MEM/WB. IR} \leftarrow \text{EX/MEM. IR};$ $\text{MEM/WB. LMD} \leftarrow$ $\text{Mem}[\text{EX/MEM. ALU}_o];$ 或 $\text{Mem}[\text{EX/MEM. ALU}_o] \leftarrow$ $\text{EX/MEM. B};$	
WB	$\text{Regs}[\text{MEM/WB. IR}[\text{rd}]] \leftarrow$ $\text{MEM/WB. ALU}_o;$ 或 $\text{Regs}[\text{MEM/WB. IR}[\text{rt}]] \leftarrow$ $\text{MEM/WB. ALU}_o;$	$\text{Regs}[\text{MEM/WB. IR}[\text{rt}]] \leftarrow$ $\text{MEM/WB. LMD};$	

3.5 流水线的实现

3. 流水线的控制

- 主要是如何控制四个多路选择器MUX。

- **MUX2**

- ```
if (ID/EX.IR[op]==“分支指令”)
```

- ```
{ MUX2_output=ID/EX.NPC };
```

- ```
else MUX2_output=ID/EX.A;
```

- ```
//MUX2_output表示MUX2的输出
```

- **MUX3**

- ```
if (ID/EX.IR[op]==“寄存器—寄存器型ALU指令”)
```

- ```
{ MUX3_output=ID/EX.B };
```

- ```
else MUX3_output=ID/EX.Imm;
```

- ```
//MUX3_output表示MUX3的输出
```

3.5 流水线的实现

□ MUX1

```
if ( (ID/EX.IR[op]==“分支指令” ) & EX/MEM.cond )  
{ MUX1_output=EX/MEM.ALUo };  
else MUX1_output=PC+4;
```

//MUX1_output表示MUX1的输出

□ MUX4

```
if (ID/EX.IR[op]==“load”)  
{ MUX4_output=MEM/WB.LMD };  
else MUX4_output=MEM/WB.ALUo
```

//MUX4_output表示MUX4的输出

3.5 流水线的实现

- ❑ **第5个多路器：**从MEM/WB回传至通用寄存器组的写入地址应该是从MEM/WB. IR[rd]和MEM/WB. IR[rt]中选一个。
 - 寄存器—寄存器型ALU指令：选择MEM/WB. IR[rd]；
 - 寄存器—立即数型ALU指令和load指令：选择MEM/WB. IR[rt]。

➤ 解决数据冲突的问题

- ❑ 所有的数据冲突均可以在ID段检测到。

如果存在数据冲突，就在相应的指令流出ID段之前将之暂停。

完成该工作的硬件称为流水线的互锁机制。

3.5 流水线的实现

- ❑ 在ID段确定需要什么样的定向，并设置相应的控制。
降低流水线的硬件复杂度。（不必挂起已经改变了机器状态的指令）
- ❑ 也可以在使用操作数的那个时钟周期的开始检测冲突和确定必需的定向。
- ❑ 检测冲突是通过比较寄存器地址是否相等来实现的。

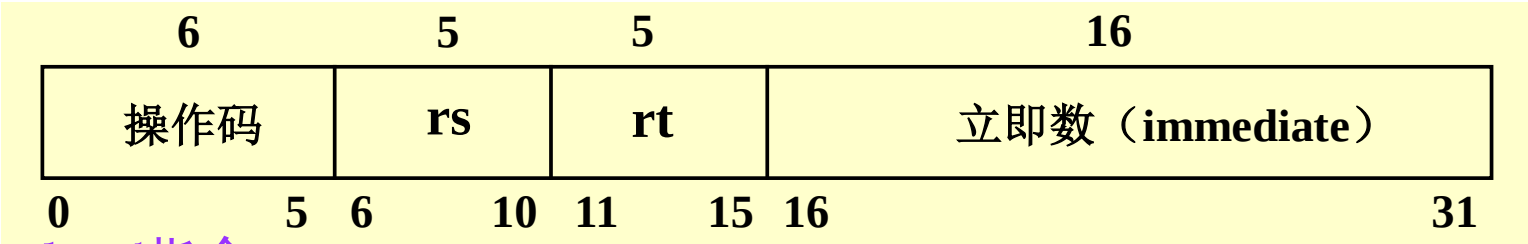
举例： load互锁

由于使用load的结果而引起的流水线互锁称为load互锁。

3.5 流水线的实现

在ID段检测是否存在Load引起的RAW冲突（这时load指令在EX段）

类指令



load指令：

访存有效地址： $\text{Regs}[\text{rs}] + \text{immediate}$
从存储器取来的数据放入寄存器 rt

ID/EX中的操作码 (ID/EX. IR[op])	IF/ID中的操作码 (IF/ID. IR[op])	比较的操作数字段
load	RR ALU	ID/EX. IR[rt]=IF/ID. IR[rs] OR ID/EX. IR[rt]=IF/ID. IR[rt]
load	load、store ALU立即数或分支	ID/EX. IR[rt]=IF/ID. IR[rs]

3.5 流水线的实现

- ❑ 若检测到RAW冲突，流水线互锁机制必须在流水线中插入停顿，并使当前正处于IF段和ID段的指令不再前进。
 - 将ID/EX. IR中的操作码改为全0
(全0表示空操作)
 - IF/ID寄存器的内容回送到自己的入口

3.5 流水线的实现

➤ 定向逻辑

- 要考虑的情况更多
- 通过比较流水寄存器中的寄存器地址来确定

例如：

- 若： $(ID/EX. IR. op == RR_ALU) \& (EX/MEM. IR. op == RR_ALU) \& (ID/EX. IR[rt] == EX/MEM. IR[rd])$

则：EX/MEM. ALU_o定向到ALU的下面一个输入

- 若： $(ID/EX. IR[op] == RR_ALU) \& (MEM/WB. IR[op] == load) \& (ID/EX. IR[rt] == MEM/WB. IR[rt])$

则：把MEM/WB. LMD定向到ALU的下面一个输入

3.5 流水线的实现

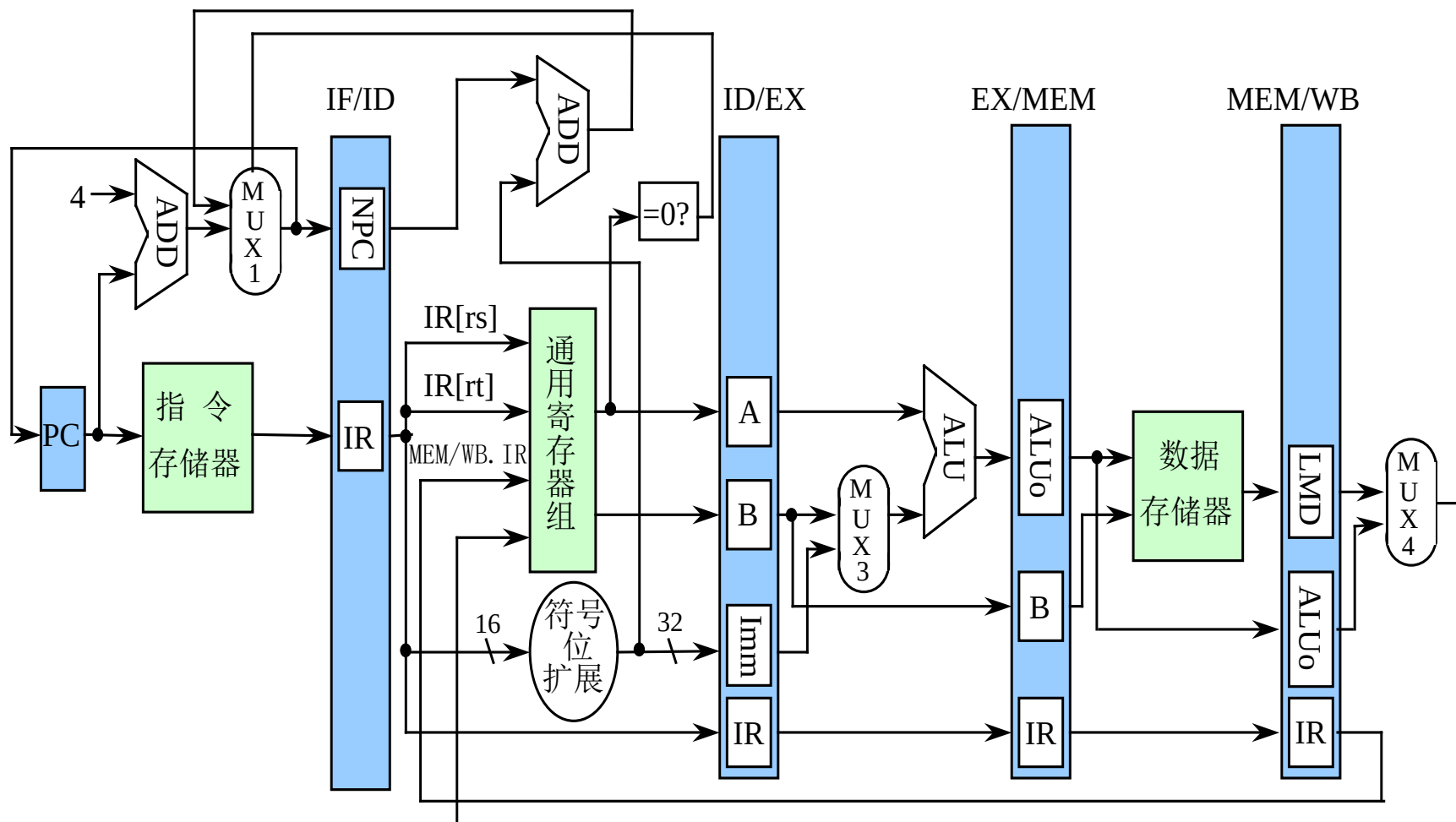
4. 控制冲突

- 分支指令的条件测试和分支目标地址计算是在EX段完成，对PC的修改是在MEM段完成。
- 它所带来的分支延迟是3个时钟周期。
- 减少分支延迟：

（把上述工作提前到ID段进行）

- 在ID段增设一个加法器：计算分支目标地址
- 把条件测试“=0?”的逻辑电路移到ID段
- 这些结果直接回送到IF段的MUX1
- 改进后的流水线对分支指令的处理

3.5 流水线的实现



为减少分支延迟，改进后的流水线数据通路

3.5 流水线的实现

改进后流水线的分支操作

流水段	分支指令操作
IF	$\begin{aligned} & \text{IF/ID. IR} \leftarrow \text{Mem}[\text{PC}]; \\ & \text{IF/ID. NPC, PC} \leftarrow \\ & \quad (\text{if}((\text{IF/ID[op]} = \text{branch}) \& ((\text{Regs}[\text{IF/ID. IR}[\text{rs}]] = 0))) \\ & \quad \{ \text{IF/ID. NPC} + (\text{IF/ID. IR}_{16})^{16} \#\# (\text{IF/ID. IR}_{16..31} \ll 2) \} \\ & \quad \text{else } \{ \text{PC} + 4 \}); \end{aligned}$
ID	$\begin{aligned} & \text{ID/EX. A} \leftarrow \text{Regs}[\text{IF/ID. IR}[\text{rs}]]; \quad \text{ID/EX. B} \leftarrow \text{Regs}[\text{IF/ID. IR}[\text{rt}]]; \\ & \text{ID/EX. IR} \leftarrow \text{IF/ID. IR}; \\ & \text{ID/EX. Imm} \leftarrow (\text{IF/ID. IR}_{16})^{16} \#\# \text{IF/ID. IR}_{16..31}; \end{aligned}$
EX	
MEM	
WB	

3.1 流水线的基本概念

时空图、分类

3.2 流水线的性能指标

吞吐率、加速比和效率

3.3 非线性流水线的调度

预约表、禁止表、冲突向量、状态转移图

小结

3.4 流水线的相关与冲突

5段RISC指令流水线

相关：数据相关、名相关、控制相关

冲突：结构冲突

数据冲突（RAW、WAW、WAR 定向技术）

控制冲突（分支延迟）

调度：分支预测、延迟槽



3.5 流水线的实现

IF、ID、EX、MEM、WB

【题1】5段指令流水线在两种处理器下各段所需时间如下：

	IF	ID	EX	MEM	WB
a	300ps	400ps	350ps	550ps	100ps
b	200ps	150ps	100ps	190ps	140ps

假设段间额外损耗均为20ps。

1、如果不使用流水线，那么周期是多少？

一条指令的执行时间是多少？吞吐率是多少？

2、如果采用流水线，那么周期是多少？

一条指令的执行时间是多少？吞吐率是多少？

3、如果你能够把一段分成时间相同的两段，那么你会怎么选择？并求新的周期、一条指令的执行时间以及吞吐率。

4、假设运行的程序指令分布如下：

ALU: 50%, BEQ: 25%, LW: 15%, SW: 10%

假设没有气泡和冲突，那么数据存储器的使用率和寄存器写端口的使用率。

解：

1、

a. $CT = 300 + 400 + 350 + 550 + 100 = 1700\text{ps}$

$$\text{Latency} = 1700\text{ps}$$

$$Tp = 1/1700 \text{ inst/ps}$$

b. $CT = 200 + 150 + 100 + 190 + 140 = 780\text{ps}$

$$\text{Latency} = 780\text{ps}$$

$$Tp = 1/780 \text{ inst/ps}$$

2、

a. $CT = 550 + 20 = 570\text{ps}$

$$\text{Latency} = 5 * 570 = 2850\text{ps}$$

$$Tp = 1/570 \text{ inst/ps}$$

b. $CT = 200 + 20 = 220\text{ps}$

$$\text{Latency} = 5 * 220 = 1100\text{ps}$$

$$Tp = 1/220 \text{ inst/ps}$$

3、

a. $CT = 400 + 20 = 420\text{ps}$
 $\text{Latency} = 6 * 420 = 2520\text{ps}$
 $Tp = 1/420 \text{ inst/ps}$

b. $CT = 190 + 20 = 210\text{ps}$
 $\text{Latency} = 6 * 210 = 1260\text{ps}$
 $Tp = 1/210 \text{ inst/ps}$

4、

数据存储器利用率 (LW+SW) : $15\% + 10\% = 25\%$
寄存器写端口利用率 (ALU+LW) : $50\% + 15\% = 65\%$

【题2】 某流水线处理器可以2GH频率运行5段流水线：即取指（IF）、译码（ID）、执行（EX）、访存（MEM）、写回（WB）。流水线的每段除EX以外，需要1个时钟周期。EX段中ADD和SUB需要一个时钟周期，MUL需要三个时钟周期，DIV需要2个时钟周期。

考虑如下指令：

I1: ADD R1, R2, R3; //R1<-R2+R3

I2: SUB R3, R2, R1; //R3<-R2-R1

I3: MUL R4, R1, R2; //R4<-R1*R2

I4: DIV R3,R4, R3; //R3<-R4/R3

要求：

1. 说明这段代码中真数据相关有哪几处？
2. 画出采用数据定向技术时流水线时空图，并计算相应的执行时间。

习题讲解

解：

	1	2	3	4	5	6	7	8	9	10	11
I1:ADD R1, R2, R3	IF	ID	EX	ME	WB						
I2:SUB R3, R2, R1		IF	ID	EX	ME	WB					
I3:MUL R4, R1, R2			IF	ID	EX	EX	EX	ME	WB		
I4:DIV R3, R4, R3				IF	-	-	ID	EX	EX	ME	WB

真数据相关：I1和I2，I3和I4 两处。

1个时钟周期= $1/2\text{GHz}(\text{秒}) = 0.5 \times 10^{-9} (\text{s}) = 0.5(\text{ns})$

总执行时间11个时钟周期，执行时间= $11 \times 0.5 = 5.5 (\text{ns})$

【题3】 一个处理器的指令流水线分为五段：取指（IF）、译码（ID）、取操作数（OF）、执行（EX）、写操作数（WO）。如果IF、ID、OF和WO各段需要1个时钟周期，EX段的ADD和SUB指令需要1个时钟周期，MUL需要3个时钟周期，DIV需要6个时钟周期。不采用定向技术，执行如下指令，画出流水线时空图，并求需要的时钟周期数。

I0:	MUL R2, R0, R1	//R2←R0*R1
I1:	DIV R5, R3, R4	//R5←R3/R4
I2:	ADD R2, R5, R2	//R2←R5+R2
I3:	SUB R5, R2, R6	//R5←R2-R6

习题讲解

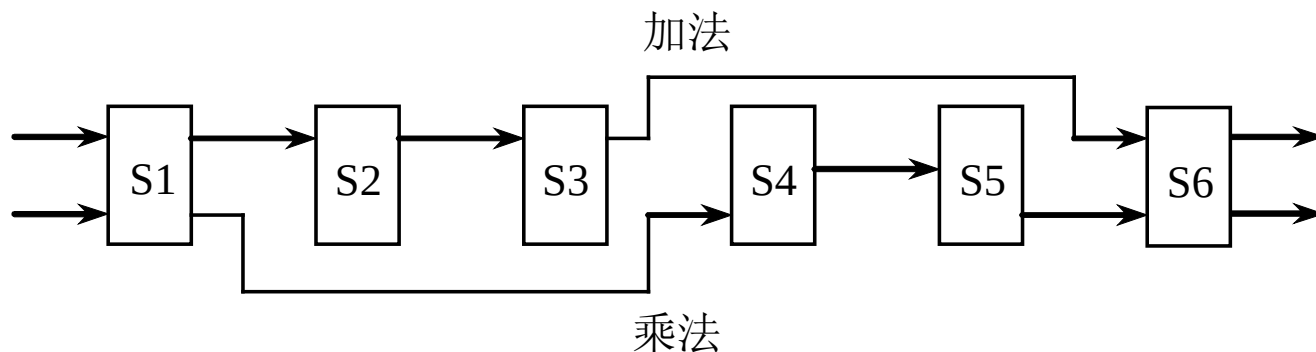
解：

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
MUL R2, R0, R1	IF	ID	OF	EX	EX	EX	WO								
DIV R5, R3, R4		IF	ID	OF	EX	EX	EX	EX	EX	EX	EX	WO			
ADD R2, R5, R2			IF	ID	—	—	—	—	—	—	OF	EX	WO		
SUB R5, R2, R6				IF	—	—	—	—	—	—	ID	—	OF	EX	WO

故需要15个时钟周期。

习题讲解

【题4】动态多功能流水线由6个功能段组成，如下图：



$$\sum_{i=1}^5 x_i y_i z_i$$

其中，S1、S4、S5、S6组成乘法流水线，S1、S2、S3、S6组成加法流水线，各个功能段时间均为50ns，假设该流水线的输出结果可以直接返回输入端，而且设置有足够的缓冲寄存器，若以最快的方式用该流水计算：

- 画出时空图；
- 计算流水线的吞吐率、加速比和效率。

习题讲解

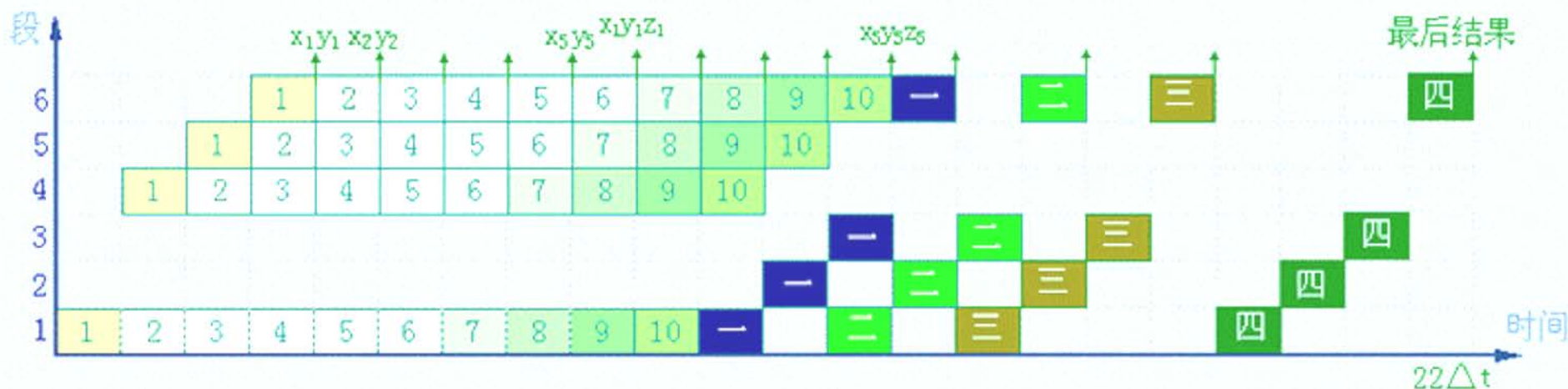
解：一共需要做10次乘法，4次加法。

$$\text{吞吐率TP} = 14 / (22 * \Delta t) = 14 / 22 / 50 * 10^9 = 12.7 \text{ M条/秒}$$

$$\text{加速比} = 14 * 4 * \Delta t / (22 * \Delta t) = 14 * 4 / 22 = 2.55$$

$$\text{效率} = 14 * 4 / (22 * 6) = 42.42\%$$

(1)



(2)

$$TP = \frac{14}{22\Delta t}$$

